
Introduction to Algorithm Engineering

Spring 2026

Lecture 3

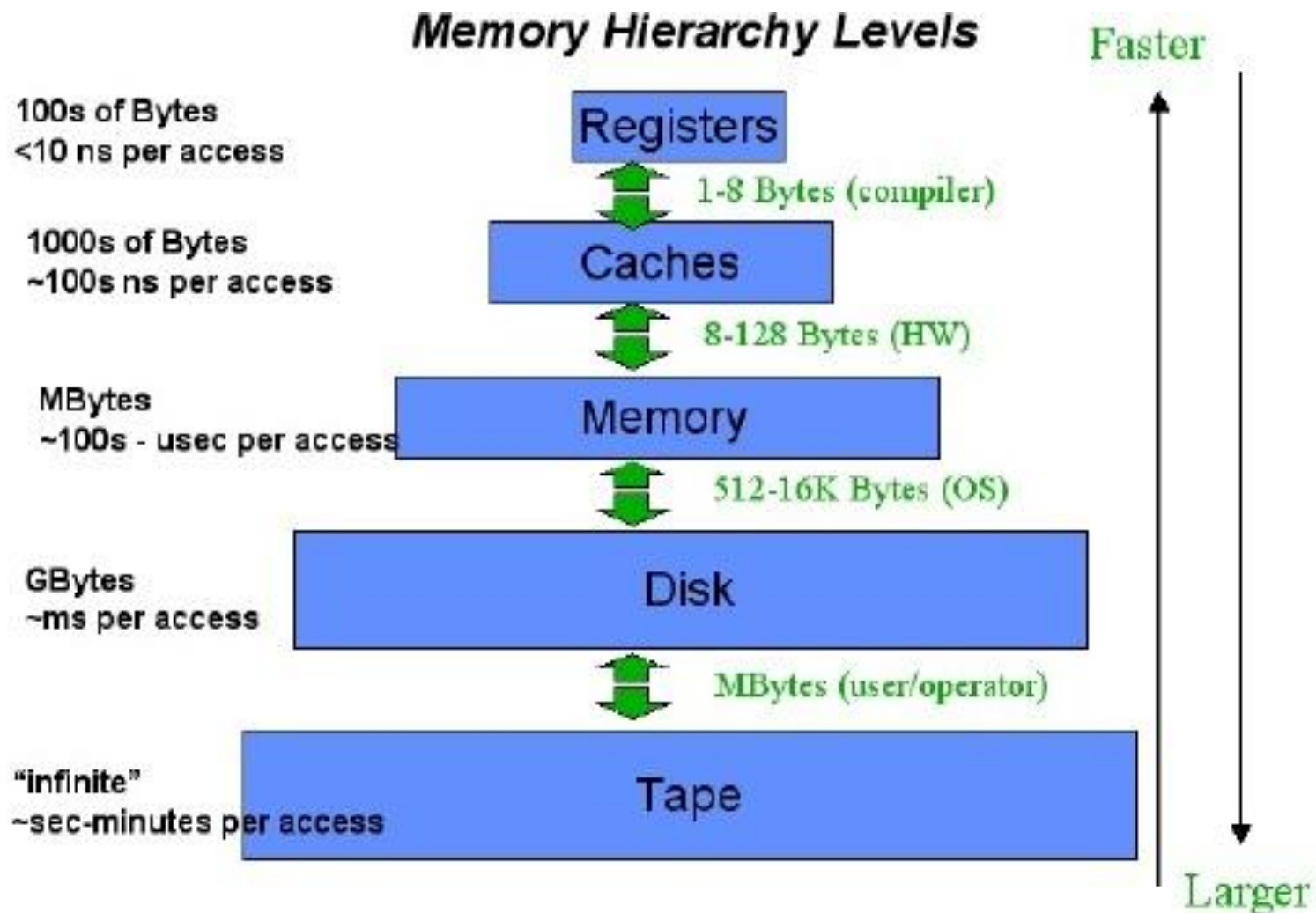
This Lecture

- In this lecture, we will study algorithm implementation from an architecture friendliness point of view.
- The central question to answer is: **How Far Should Algorithms Consider the Architecture?**
- There are predominantly two schools of thought
 - Very much – Cache Aware Algorithm Design
 - Should never – Cache Oblivious Algorithm Design
- Both sides have good arguments.
- We will study both sides.

Cache Aware Algorithm Design

- A model that is relevant to efficient implementations of algorithms.
- Models the twin challenges of algorithm design for big data and a deep memory hierarchy.
- For instance, consider the case that the entire data does not fit in the memory of a computer.
- Then, the assumption that all memory references require the same access time is not truly valid.
- How to design and implement algorithms when the above assumption does not hold?

Example Hierarchy



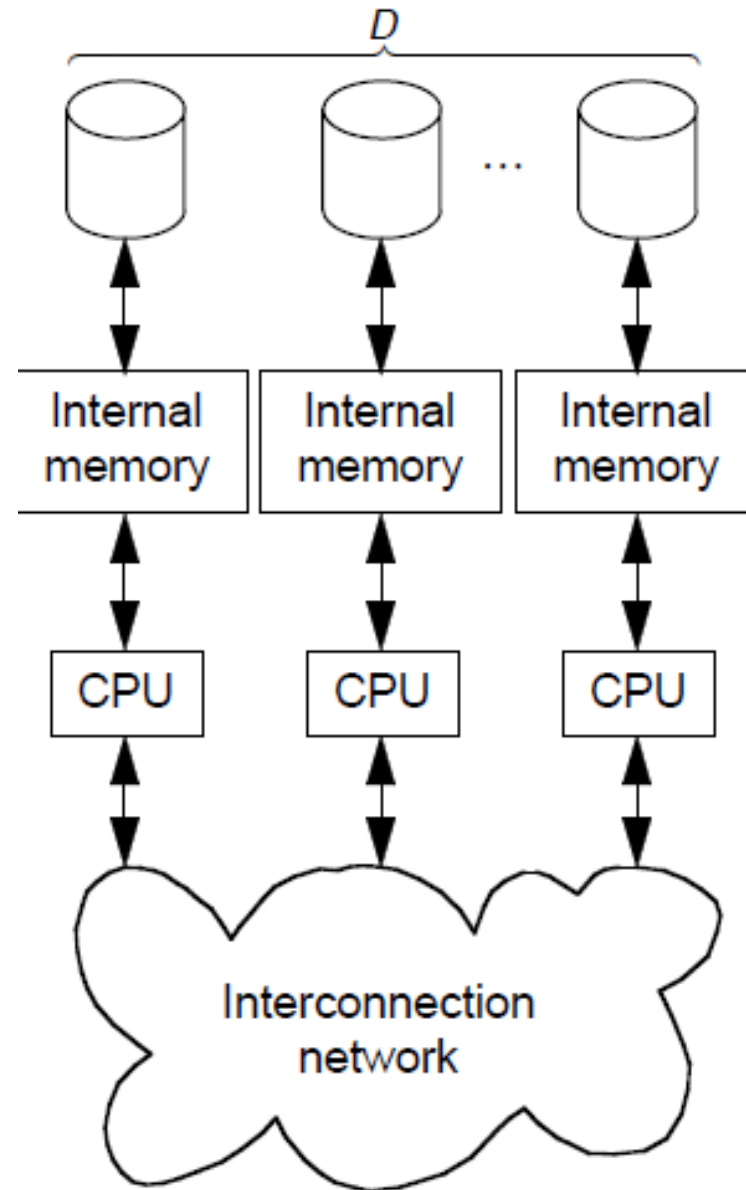
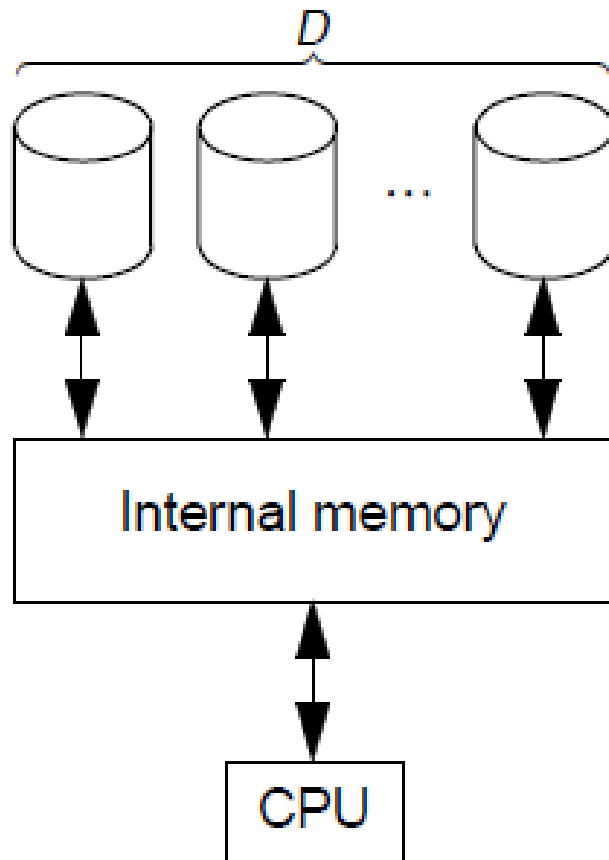
Some Definitions

- **Working Set:** A collection of data items recently accessed by a program.
- A small working set is good. Can fit in the cache, and a prefetcher can help in most cases minimize cache misses.
- However, prefetch works best for computations with very strong locality in the computation.
 - Example: Add elements of two arrays index-wise.
- But, there are other computations that have strong non-local access patterns.
- In such cases, explicit memory management helps.

Cache-Aware Algorithms

- One school of thought is to design algorithms based on the memory hierarchy of the target architecture.
- Called as **Cache-Aware Algorithms**.
- Consider the setting where there are two levels of memory
- The transfer unit from the larger memory to the smaller memory is B .

Model in Pictures



Performance Parameters

- The number of I/O operations,
 - The disk space usage, and
 - The CPU time.
-
- N = problem size (in units of data items);
 - M = internal memory size (in units of data items);
 - B = block transfer size (in units of data items);
-
- Notice that $M < N$, and $1 \leq B \leq M/2$

Performance Parameters

- The number of I/O operations,
 - The disk space usage, and
 - The CPU time.
-
- N = problem size (in units of data items);
 - M = internal memory size (in units of data items);
 - B = block transfer size (in units of data items);
 - D = number of independent disk drives;
 - P = number of CPUs
-
- Notice that $M < N$, and $1 \leq DB \leq M/2$

Four Fundamental Operations

- **Scanning** (a.k.a. streaming or touching) a file of N data items, which involves reading or writing the items in the file sequentially.
- **Sorting** a file of N data items, which puts the items into sorted order.
- **Searching** online through N sorted data items.
- The fourth one, is on **batched query** style operations. Read from Vitter's survey.

Scanning

- The I/O bound $\text{Scan}(N) = O(N/B)$.
- Relatively straight-forward to understand.
- Sorting requires some additional tricks.

Searching

- Refer to Question 1 in the worksheet.

Cache-Aware Algorithms

- Consider the setting where there are two levels of memory
- The transfer unit from the larger memory to the smaller memory is B .

Matrix Multiplication

- A good case study of how to tune algorithms to a cache size.

```
MatrixMultiply(A, B, C, n)
Begin
  for i = 1 to n do
    for j = 1 to n do
      for k = 1 to n do
        C[i,j] += A[i,k]xB[k,j]
      end-for
    end-for
  end-for
End.
```

- Asymptotic run time is $O(n^3)$ where the matrices are of size $n \times n$.

Matrix Multiplication

- A good case study of how to tune algorithms to a cache size.

```
MatrixMultiply(A, B, C, N)
Begin
  for i = 1 to N do
    for j = 1 to N do
      for k = 1 to N do
        C[i,j] += A[i,k]xB[k,j]
      end-for
    end-for
  end-for
End.
```

- Suffers from lots of cache misses.
- Exercise: Estimate the number of cache misses.

Matrix Multiplication – Naive

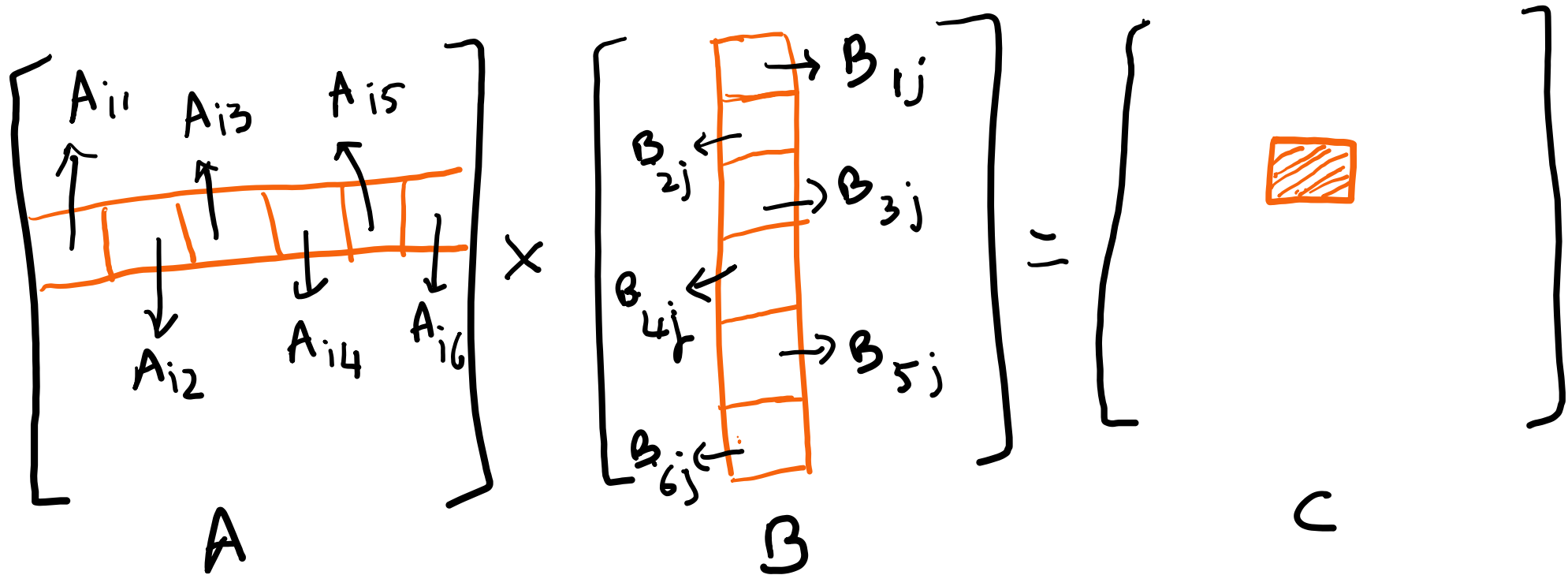
- To compute each element of C:
 - Read/Scan the entire row of A – $O(N/B)$ cache misses
 - Read/Scan the column of B – $O(N)$ cache misses.
 - Why?
 - Cache misses per element of C = $O(N) + O(N/B) = O(N)$.
- Overall cache misses = $O(N^3)$ as we need to compute N^2 elements of the matrix C.

Matrix Multiplication

```
MatrixMultiply(A, B, C, n)
Begin
  for i = 1 to n/s do
    for j = 1 to n/s do
      for k = 1 to n/s do
         $C_{ij} = C_{ij} + \text{Mul}(A_{ik}, B_{kj}, s)$ 
      end-for
    end-for
  end-for
End.
```

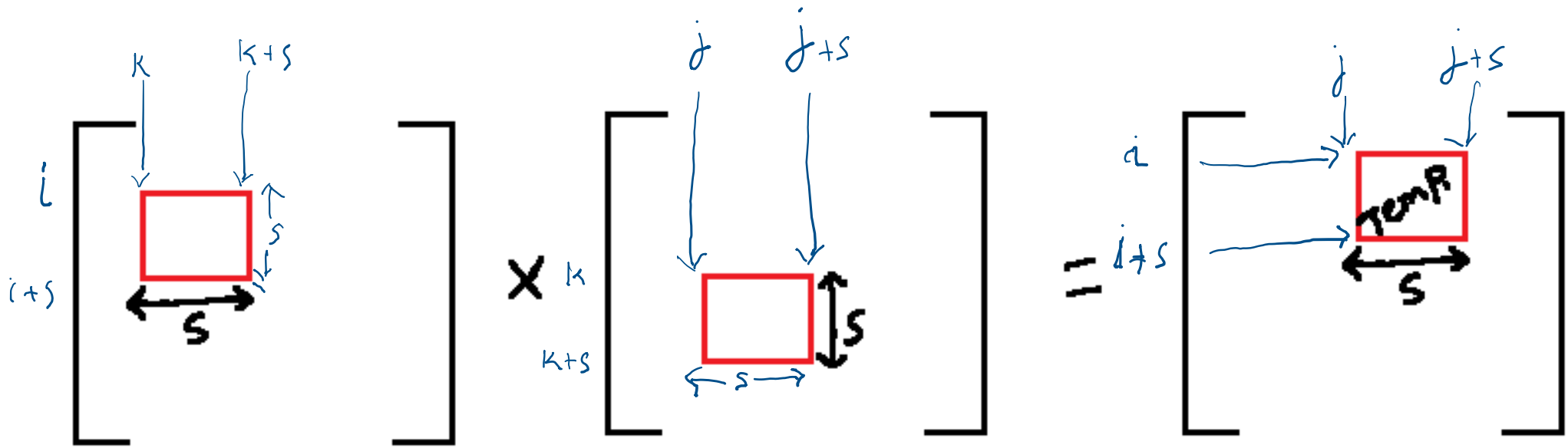
- It is however better to imagine that the computation be broken into blocks of $s \times s$ submatrices.

Matrix Multiplication in Pictures



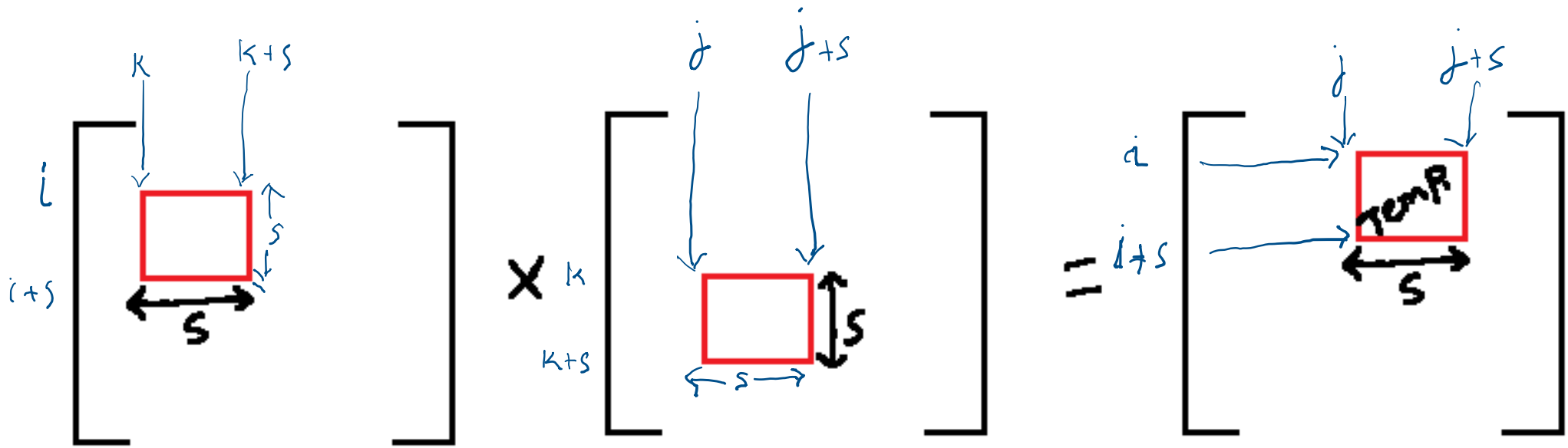
$$C_{ij} = A_{i1} \times B_{1j} + A_{i2} \times B_{2j} + A_{i3} \times B_{3j} + A_{i4} \times B_{4j} + A_{i5} \times B_{5j} + A_{i6} \times B_{6j}$$

Matrix Multiplication in Pictures



$$\begin{aligned}
 C_{ij} = & A[i:i+s:1:s] \times B[1:s;j:j+s] + \\
 & A[i:i+s;s+1:2s] \times B[s+1:2s;j:j+s] + \\
 & \vdots \\
 & A[i:i+s:n-s:n] \times B[n-s:n;j:j+s]
 \end{aligned}$$

Matrix Multiplication in Pictures



$$\begin{aligned}
 C_{ij} = & A[i:i+s:1:s] \times B[1:s;j:j+s] + \\
 & A[i:i+s;s+1:2s] \times B[s+1:2s;j:j+s] + \\
 & \vdots \\
 & A[i:i+s:n-s:n] \times B[n-s:n;j:j+s]
 \end{aligned}$$

Improved Version

- Estimate the number of cache misses again.
- To compute a $s \times s$ submatrix of C , we need to bring in $O(s^2)$ elements into the cache.
- Results in $O(s^2/B)$ cache misses.

Improved Version

- Estimate the number of cache misses again.
- To compute a $s \times s$ submatrix of C , we need to bring in $O(s^2)$ elements into the cache.
- Results in $O(s^2/B)$ cache misses.
- For the entire C matrix, the number of cache misses = $O(s^2/B) \times (n/s)^3 = O(n^3/sB)$.
 - Why?

Improved Version

- Estimate the number of cache misses again.
- To compute an $s \times s$ submatrix of C , we need to bring in $O(s^2)$ elements into the cache.
- Results in $O(s^2/B)$ cache misses.
- For the entire C matrix, the number of cache misses = $O(s^2/B) \times (n/s)^3 = O(n^3/sB)$.
 - Why?
 - The program has a three way nested loop of n/s iterations in each loop.

Improved Version

- Estimate the number of cache misses again.
- To compute an $s \times s$ submatrix of C , we need to bring in $O(s^2)$ elements into the cache.
- Results in $O(s^2/B)$ cache misses.
- For the entire C matrix, the number of cache misses = $O(s^2/B) \times (n/s)^3 = O(n^3/sB)$.
- Tall cache Assumption: $M = \Omega(B^2)$, i.e., many more cache lines fit in the cache.
- Number of cache misses = $O(n^3/BM^{1/2})$ at $s = B$.

Improved Version

- Estimate the number of cache misses again.
- Each C_{ij} involves the transfer of $O(s^2)$ items of A, B, and C.
- This results in $O(s^2/B)$ cache misses.
- There are a total of $(n/s)^3$ iterations.
- Hence, the total number of cache misses = $(n/s)^3 \times O(s^2/B) = O(n^3/sB)$.

```
MatrixMultiply(A, B, C, n)
Begin
  for i = 1 to n/s do
    for j = 1 to n/s do
      for k = 1 to n/s do
         $C_{ij} = C_{ij} + \text{Mul}(A_{ik}, B_{kj}, s)$ 
      end-for
    end-for
  end-for
End.
```